



LE LANGAGE PYTHON

STRUCTURES DE DONNÉES

LISTES - TUPLES — DICTIONNAIRES - SETS

CRMEF — MARRAKECH

- DÉFINITION
- CRÉATION DES LISTES
- PARCOURS D'UNE LISTE
- MODIFICATION D'UNE LISTE
- RECHERCHE DANS UNE LISTE
- LISTES SONT DES OBJETS
- COPIE D'UNE LISTE

LES LISTES

Python

- **Définition d'une liste**

- Une liste est une collection ordonnée et modifiable d'éléments éventuellement hétérogènes.

- Exemples:

```
>>> nombres=[1,2,3,4,5]
>>> jours=['lundi', 'mardi', 'mercredi', 'jeudi']
>>> alphanum=[1, 'janvier', 2, 'février', 3, 'mars', [4, 'avril']]
>>> alphanum
[1, 'janvier', 2, 'février', 3, 'mars', [4, 'avril']]
>>>
```

- **Création d'une liste**

- list(), [], range,...

- **Accès aux éléments d'une liste**

- Pour accéder aux éléments d'une liste, on utilise les mêmes méthodes (index, découpage en tranches) que pour accéder aux caractères d'une chaîne :

- Exemples:

```
>>> alphanum[3]
'février'
>>> alphanum[-1]
[4, 'avril']
>>> |
```

LES LISTES

Python

- Parcours d'une liste à l'aide de for, range() et len()

```
for i in mois:  
    print(i)
```

```
mois=['janvier', 'février', 'mars', 'avril']  
for index in range(len(mois)):  
    print(index, mois[index])
```

LES LISTES

Python

- **Les listes sont modifiables**
 - Les listes sont des *séquences modifiables*. Cela nous permettra de construire plus tard des listes de grande taille, morceau par morceau, d'une manière dynamique
- **Exemples :**

```
>>> alphanum[-1]=4
>>> alphanum
[1, 'janvier', 2, 'février', 3, 'mars', 4]
>>> |
```

LES LISTES

Python

- **Techniques de slicing avancé pour modifier une liste**
- Vous pouvez utiliser le « découpage en tranches » (*slicing*) pour ajouter ou supprimer des éléments dans une liste à l'aide du seul opérateur [].
- **a- Insertion d'un ou plusieurs éléments n'importe où dans une liste**
- **Exemples :**

```
>>> alphanum=[1, 'janvier', 2, 'février', 3, 'mars', 5, 'mai']
>>> alphanum[6:6] =[4, 'avril'] #alphanum=alphanum[:6]+[4]+alphanum[6:]
>>> alphanum
[1, 'janvier', 2, 'février', 3, 'mars', 4, 'avril', 5, 'mai']
```

b- Suppression/remplacement d'éléments

- **Exemples :**

```
>>> alphanum[2:4]=[] #[] désigne une liste vide
>>> alphanum
[1, 'janvier', 3, 'mars', 4, 'avril', 5, 'mai']
>>> alphanum[0:2]=[2, 'février']
>>> alphanum
[2, 'février', 3, 'mars', 4, 'avril', 5, 'mai']
```

LES LISTES

Python

- **Les listes sont des objets**
 - Sous Python, les listes sont des objets pour les quels on peut appliquer un certain nombre de méthodes (fonctions).

```
>>> nombres.sort()
>>> nombres.sort(reverse=True)
>>> nombres
[5, 4, 3, 2, 1]
>>> alphanum.append('avril')
>>> alphanum
[1, 'janvier', 2, 'février', 3, 'mars', 4, 'avril']
>>> alphanum.index('avril')
7
>>> alphanum.remove('avril')
>>> del nombres[1:3]
>>> nombres
[5, 2, 1]
```

LES LISTES

Python

- Opérations sur les listes
- Concaténation : +
- Multiplication : *

```
>>> nombres=['entier', 'réel']
>>> logique=['True', 'False']
>>> nombres+logique
['entier', 'réel', 'True', 'False']
>>> nombres*2
['entier', 'réel', 'entier', 'réel']
```

- Copie d'une liste

- DÉFINITION
- CRÉATION DES TUPLES
- PARCOURS D'UN TUPLE
- RECHERCHE DANS UN TUPLE
- OPÉRATION SUR LES TUPLES
- DIFFÉRENCE ENTRE LISTE ET TUPLE

LES TUPLES

Python

- **Définition**
- Un tuple est un type de données composite, qui est assez semblable a une liste mais qui, comme les chaines, n'est pas modifiable.
- **Syntaxes :**

```
>>> ligne='p', 'y', 't', 'h', 'o', 'n'
>>> ligne
('p', 'y', 't', 'h', 'o', 'n')
>>> #ou bien
>>> ligne=('p', 'y', 't', 'h', 'o', 'n')
>>> ligne
('p', 'y', 't', 'h', 'o', 'n')
```

- **Opérations sur les tuples**
- On peut effectuer sur les tuples toutes les opérations des listes, sauf celles qui permettent la modification (del, remove(), ...)

- DÉFINITION
- CRÉATION DES DICTIONNAIRES
- PARCOURS D'UN DICTIONNAIRE
- RECHERCHE DANS UN DICTIONNAIRE
- OPÉRATION SUR LES DICTIONNAIRES

LES DICTIONNAIRES

Python

- **Définition**
- Les dictionnaires constituent un autre type de données composite, qui contiennent des éléments **non-ordonnés** et qui sont indexés par des **clés**.
- Ils sont modifiables comme les listes.
- Comme dans une liste, les éléments d'un dictionnaire sont : les valeurs numériques, les chaînes, les listes, les tuples, les dictionnaires, les fonctions, les classes ou les instances
- **Création d'un dictionnaire {}:**

```
>>> serie={} #création d'un dict vide
>>> serie['un']=1 #ajout des éléments
>>> serie['deux']=2
>>> serie['trois']=3
>>> |
```

LES DICTIONNAIRES

Python

- Opérations sur les dictionnaires

```
>>> serie['quatre']=4 #ajout d'une nouvelle paire clé/valeur
>>> serie
{'quatre': 4, 'un': 1, 'trois': 3, 'deux': 2}
>>> del serie['quatre']
>>> serie
{'un': 1, 'trois': 3, 'deux': 2}
>>> |
```

- Test d'appartenance :

```
if 'deux' in serie:
    print ('existe')
else:
    print ('n\'existe pas')
|
```

LES DICTIONNAIRES

Python

- Quelques méthodes
 - Keys(), values(), items(), copy()

```
>>> serie.keys()
dict_keys(['un', 'trois', 'deux'])
>>> serie.values()
dict_values([1, 3, 2])
>>> serie.items()
dict_items([('un', 1), ('trois', 3), ('deux', 2)])
>>> nbre=serie.copy()#créé une vraie copie
>>> nbre['quatre']=4
>>> nbre
{'quatre': 4, 'un': 1, 'deux': 2, 'trois': 3}
>>> serie
{'un': 1, 'trois': 3, 'deux': 2}
```

- Parcours d'un dictionnaire :
 - au cours de l'itération, ce sont les clés utilisées dans le dictionnaire qui seront successivement affectées à la variable de travail, et non les valeurs ;
 - l'ordre dans lequel les éléments seront extraits est *imprévisible* (puisque un dictionnaire n'est pas une séquence).

- DÉFINITION
- CRÉATION DES ENSEMBLES
- OPÉRATION SUR LES ENSEMBLES

- **Ensembles (SETS)**

- Les objets de type « ensemble » (**set** dans le langage Python) sont des structures de données qui modélisent la notion mathématique d'ensemble. Un tel objet peut contenir des valeurs de type quelconque, mais une même valeur ne peut apparaître qu'une seule fois. De plus les éléments d'un objet de type ensemble ne sont pas ordonnés (on ne peut pas y accéder par un indice : on peut juste savoir si une valeur est ou n'est pas élément de l'ensemble).
- On forme un ensemble par la séquence de ses éléments (valeurs), encadrée par { et }, ou en utilisant le constructeur **set**

- **Exemple**

- $E = \{1,5,2,3,2,7,5,2,1,3,2\}$
- `E= set([1,5,2,3,2,7,5,2,1,3,2])`
 - #même résultat en convertissant une liste
 - # dans un ensemble, tous les doublons ont été éliminés

COMPARAISON ENTRE LES STRUCTURES DE DONNÉES

Python

LISTES

- Modifiable
- Structure de données très utilisée
- Demande plus de mémoire que les tuples
- Séquence
- Tri

TUPLES

- Immuable
- Utile pour les données fixes
- Plus rapide que les listes
- Séquence

DICTIONNAIRES

- Paires clé/valeur
- Tableaux associatifs
- Ne sont pas ordonnés

ENSEMBLES (SETS)

- Stocke des éléments non dupliqués
- Op ensemblistes Math
- Ne sont pas ordonnés